A Derived-Key Security Layer for Opportunistic Bluetooth Mesh Bubbles: Specification and Analysis

BUBBLE-BLUE Project — Security Layer Specification

Companion to the systems preprint [1]

Working draft — July 20, 2026

Abstract

We specify the security layer of BUBBLE-BLUE, a leaderless, infrastructure-free Bluetooth mesh for closed groups (*bubbles*) communicating over multi-hop opportunistic links. The layer provides group confidentiality, per-packet authenticated encryption, replay resistance, private member-to-member channels, and in-band membership revocation, at a fixed 42-byte per-packet overhead. Its organizing principle is that *no key material ever travels on data packets*: every packet key is derived locally from a shared secret and public header fields via HKDF-SHA256, and pairwise keys are obtained by X25519 key agreement rather than distribution. Membership changes are unified under a monotone *epoch*: a coordinator floods a single multi-recipient rekey message whose wrapped-key header adapts the multi-key idea of the originating preprint to the rare control path, and stragglers self-heal through epoch-tagged Hello gossip. Control-plane statements — key bindings and rekeys — carry Ed25519 attestations derived from the same identity seed. We give the complete wire formats, the encryption and revocation algorithms, an overhead account covering both implemented transports (Bluetooth Classic and BLE L2CAP CoC), a descriptive security analysis against a structured adversary model, a mapping from claimed properties to the test suite, and an explicit list of assumptions and residual limitations.

1 Introduction and Scope

BUBBLE-BLUE targets small closed groups (n in the tens) that must communicate over Bluetooth with no infrastructure: the physical topology is an opportunistic mesh, nodes relay one another's frames, and relays may not even belong to the same group. The originating preprint [1] fixes the operational requirements: tamper-proof exchange, protection of the control plane against outsiders, a fresh session key per packet, member repudiation and addition, tolerance to loss of the leader, and replay resistance.

This document specifies the security layer *as implemented*, covering both transports in production — Bluetooth Classic L2CAP (MTU 1021) and BLE L2CAP connection-oriented channels (MTU 512) — and the three extensions adopted after the security audit: network-wide public-key announcement (§6.2), the single-flood multi-recipient rekey (§7.2), and Ed25519 attestations on control-plane statements (§6.3).

Relation to the preprint's multi-key header. The preprint proposes a per-packet header carrying the session key wrapped once per recipient (16-byte key material per member, i.e. $16n$ bytes per packet), with repudiation performed by each sender locally zeroing the revoked member's slot [1]. We retain the wrapped-key header *idea* but relocate it from the frequent data path to the rare control path: the multi-recipient header appears only in the revocation flood, where its structure is exactly what is needed, while data packets carry a single 16-byte authentication tag. This preserves the four operational properties of the original repudiation — instantaneous, no blocking ceremony, absentee-tolerant, isolation of the revoked member — while eliminating its two defects for our target: an $O(n)$ permanent per-packet cost incompatible with the BLE MTU at moderate n, and a purely sender-local revocation state that can silently diverge across the group.

Contributions of this specification. (i) a complete, implementation-faithful description of the key hierarchy and packet AEAD (§3–4); (ii) the anonymous control plane, including the attested `ANNOUNCE` beacon that closes the two-hop key-learning gap (§6); (iii) the epoch mechanism with the signed multi-recipient rekey flood, receiver-side coordinator verification, and MTU-safe partition on both transports (§7); (iv) a security analysis with an adversary-coverage table and a property-to-test map (§8); (v) explicit assumptions and limitations (§10).

2 Preliminaries

2.1 Primitives

All cryptographic operations are instantiated from a single vetted, OpenSSL-backed library.

- **AEAD.** AES-256-GCM [5]. We write $c \leftarrow \mathsf{Enc}_K(N, m, A)$ for encryption of plaintext m under key K with 96-bit nonce N and associated data A; c is ciphertext with the 16-byte tag appended.

Symbol	Meaning
gk_e	32-byte group key of epoch e; gk_0 is the bootstrap PSK
R_e	set of revoked X25519 public keys at epoch e
bd	6-byte Bluetooth device address
aid	8-byte pseudonymous identifier (Eq. 1)
sk_A, pk_A	X25519 identity key pair of node A
sk_A^σ, pk_A^σ	Ed25519 signing pair (same seed)
K_{p}	pairwise key (ECDH-derived, epoch-separated)
K_{pkt}	per-packet AEAD key (Eq. 5)
K_{w}	wrap key, per message and recipient (Eq. 6)
k_{m}	one-time message key (multi-recip.)
$H,\ \bar{H}$	26-byte routing header; H with *ttl* zeroed (AAD image)
ρ, τ	header size (26 B) and GCM tag size (16 B)
W, Δ	replay window (256) and timestamp bound (120 s)

Table 1: Notation.

$\mathsf{Dec}_K(N, c, A)$ returns m or $\perp$ (`InvalidTag`) on any authentication failure.

- **KDF.** $\mathsf{HKDF}(ikm;\ s,\ info)$ is HKDF-SHA256 [2] with salt s (possibly absent) and context `info`, always producing 32 bytes.
- **Key agreement.** X25519 [3]; $\mathsf{X25519}(sk, pk)$ denotes the raw Diffie–Hellman shared secret.
- **Signatures.** Ed25519 [4]; $\mathsf{Sig}_{sk}(m)$ is a 64-byte signature, $\mathsf{Vf}_{pk}(m, \sigma) \in \{0, 1\}$ its verification.
- **PRF.** HMAC-SHA256, for pseudonymous identifiers.

2.2 Notation

Table 1 collects the symbols used throughout.

2.3 Nonce discipline

Every AEAD invocation on the data path and on sequence-numbered control paths uses the *fixed zero nonce* $N = 0^{96}$. This is safe because the encrypting key is unique per invocation: packet and wrap keys mix the tuple (src, seq, ts) into the KDF context (Eqs. 5, 6), so each key encrypts exactly one message and the (key, nonce) pair never repeats — the sole condition GCM requires [5]. Including *ts* in the derivation also covers a sequence-counter reset after reboot. The discipline saves 12 on-wire bytes per packet. The one exception is the Hello private part (§6.1), which has no sequence number and therefore prepends a fresh random nonce.

2.4 Routing header

Every packet begins with a cleartext routing header of $\rho = 26$ bytes,

$$H = (type,\ src,\ dst,\ ttl,\ seq,\ ts),$$

with *seq* a per-source `uint32` counter and *ts* a `uint64` millisecond timestamp. $\bar{H}$ denotes H with the mutable *ttl* byte set to zero, so relays may decrement *ttl* in flight without invalidating authentication — the same treatment IPsec gives mutable fields.

3 Key Hierarchy

3.1 Bootstrap, pseudonyms, identities

A single 32-byte pre-shared key gk_0 (the bubble secret) is distributed out of band before the mission, matching the preprint's trust model. All secrets are thereafter *derived*; none is transmitted.

Each node advertises an 8-byte pseudonym in place of its BD address:

$$aid(gk, bd) \ = \ \mathsf{HMAC}_{gk}(bd)\,[0{:}8]. \tag{1}$$

The recommended (file-less) identity binds a 32-byte *identity seed* to the group secret and the address:

$$seed_{bd} = \mathsf{HKDF}\big(gk;\ s_{\mathrm{id}},\ \text{“…x25519-identity:”} \,\|\, bd\big), \tag{2}$$

with s_{id} = “bb-identity-v1”. The seed yields *both* asymmetric pairs by further domain-separated derivation: $sk_{bd} = seed_{bd}$ with $pk_{bd} = sk_{bd} \cdot G$ (G the Curve25519 base point) for key agreement, and

$$sk_{bd}^\sigma = \mathsf{HKDF}\big(seed_{bd};\ s_\sigma,\ i_\sigma\big) \tag{3}$$

with s_σ = “bb-ed25519-v1” and i_σ = “bubble-blue-signing”, for Ed25519 attestations (§6.3). One persisted (or derived) secret therefore anchors encryption *and* authentication. A file-persisted random seed (mode `0600`) is also supported; §10 discusses the trade-off, which now extends to the signing keys.

3.2 Pairwise keys

For members A, B that know each other's public keys, the pairwise key at epoch e is

$$K_{\mathrm{p}_e}^{AB} = \mathsf{HKDF}\big(\mathsf{X25519}(sk_A, pk_B);\ s_{\mathrm{p}},\ I_e\big), \tag{4}$$

with s_{p} = “bubble-blue-v1” and epoch domain separation $I_e = \varepsilon$ for $e{=}0$ (wire compatibility) and $I_e = \mathrm{u32}(e)$ otherwise. ECDH symmetry gives $K_{\mathrm{p}_e}^{AB} = K_{\mathrm{p}_e}^{BA}$; the key is never transmitted, and a third member cannot compute it without one of the two private keys (§8.2).

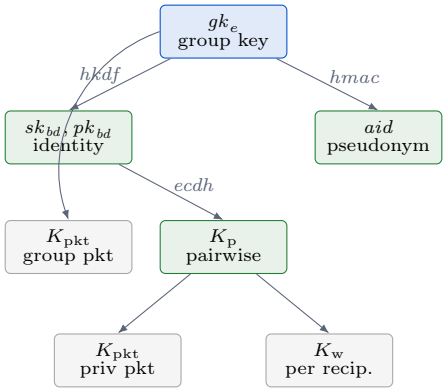


Figure 1: Key hierarchy. Everything descends from the epoch group key gk_e by local derivation; only public keys ever appear on the wire, and only in the encrypted control plane.

3.3 Per-packet and wrap keys

Data and unicast control packets derive a fresh AEAD key from a base key $K_{\text{base}} \in \{gk_e,\ K_{\text{p}}\}$:

$$K_{\text{pkt}} = \mathsf{HKDF}\big(K_{\text{base}};\ -,\ src \,\|\, \text{u32}(seq) \,\|\, \text{u64}(ts)\big). \quad (5)$$

Multi-recipient control messages wrap a one-time key k_{m} per recipient under a wrap key derived from the pairwise key with a *distinct* salt s_{w} = "bb-subgroup-wrap-v1":

$$K_{\text{w}} = \mathsf{HKDF}\big(K_{\text{p}};\ s_{\text{w}},\ src \,\|\, \text{u32}(seq) \,\|\, \text{u64}(ts)\big). \quad (6)$$

The salt separation guarantees that a wrap key can never coincide with a packet key, so the zero-nonce discipline cannot be violated across the two uses of the same pairwise secret. Figure 1 summarizes the hierarchy.

4 Packet-Level AEAD

4.1 Wire layout

A data packet is $P = H \,\|\, c$ with

$$c = \mathsf{Enc}_{K_{\text{pkt}}}\big(0^{96},\ m,\ \bar{H}\big),$$

i.e. the cleartext header followed by ciphertext-with-tag. Fixed overhead is $\rho + \tau = 42$ bytes on either transport; the maximum chat payload is 979 bytes on Bluetooth Classic (MTU 1021) and 470 bytes on BLE L2CAP CoC (MTU 512). Oversized frames on BLE are handled by an optional transport-level fragmentation layer (marker byte, per-peer reassembly), invisible to the security layer. Figure 2 shows the layout of the two packet families side by side; Table 2 lists all message types.

4.2 Send and receive

Algorithms 1 and 2 give the procedures. The receiver reconstructs K_{pkt} from the cleartext header, so no key or nonce is ever transmitted; decryption is attempted under the current group key, then the bootstrap key (tolerating epoch laggards).

Message	Scope	Base key	Signed
`HELLO` pub. part	2-hop, clear	—	—
`HELLO` priv. part	2-hop	gk_e (direct)	—
`DATA` (group)	flood	$gk_e \to K_{\text{pkt}}$	—
`DATA` (private)	routed unicast	$K_{\text{p}} \to K_{\text{pkt}}$	—
`ANNOUNCE`	network flood	$gk_e \to K_{\text{pkt}}$	PoP σ
`SUBGROUP`	flood, subset	$K_{\text{p}} \to K_{\text{w}}, k_{\text{m}}$	—
`REKEY` (gossip)	routed unicast	$K_{\text{p}} \to K_{\text{pkt}}$	state + σ
`REKEY_FLOOD`	network flood	$K_{\text{p}} \to K_{\text{w}}, k_{\text{m}}$	coord. σ

Table 2: Message types (a legacy `ACK` type is reserved and unused). Every AEAD is AES-256-GCM with zero nonce except the Hello private part (random nonce); every AAD includes $\bar{H}$; multi-recipient messages additionally bind a distinct mode marker and the case digest. Hello private v0x03 carries both pk_{bd} and pk^{σ}_{bd}.

Algorithm 1 Send data packet

Require: base key K_{base}, plaintext m, src, dst
1: $seq \leftarrow seq + 1$; $ts \leftarrow$ now_ms()
2: $H \leftarrow (type, src, dst, ttl_{\max}, seq, ts)$
3: $K_{\text{pkt}} \leftarrow \mathsf{HKDF}(K_{\text{base}}; -, src \,\|\, seq \,\|\, ts)$
4: **return** $H \,\|\, \mathsf{Enc}_{K_{\text{pkt}}}(0^{96},\ m,\ \bar{H})$

4.3 Integrity scope

The tag authenticates c and $\bar{H}$ jointly: modifying src, dst, seq, ts, $type$, or the ciphertext yields $\perp$ and a silent drop. Only ttl is malleable, by design: a relay — even a non-member — reads dst and ttl, decrements, and re-emits opaque bytes, never deriving a key.

5 Replay Protection

Three independent mechanisms interlock.

1. **Sliding window.** Per source, a filter tracks the last $W = 256$ sequence numbers, tolerating multi-hop reordering while rejecting repeats. Two instances run with separate locks — one on the relay path, one on the delivery path — so neither can mask the other.
2. **Timestamp bound.** Packets with $|ts - \text{now}| > \Delta = 120\,\text{s}$ are rejected before the window is consulted.
3. **Forward dedup.** A bounded cache (TTL 240 s, $\leq$ 4096 entries) keyed by (src, seq) suppresses duplicate relays of floods.

This strictly strengthens the preprint's "last ten timestamps". Crucially, seq and ts do double duty: they feed both the replay filter *and* the key derivation (5). An attacker who edits seq to slip past the window simultaneously changes K_{pkt} and $\bar{H}$, so authentication fails — the two mechanisms lock each other.

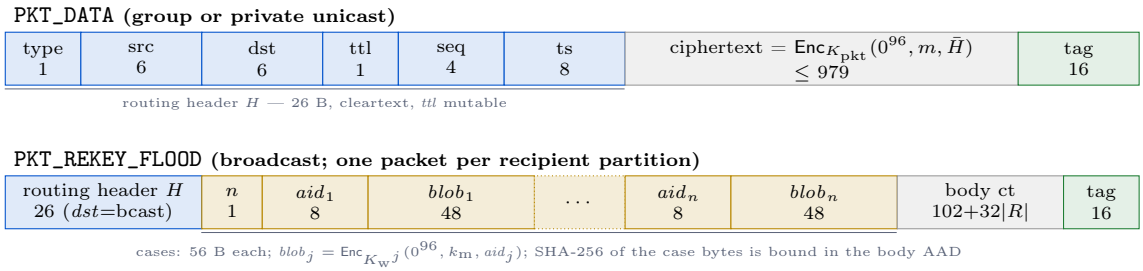


Figure 2: Wire layouts (byte counts under each field). Top: data packet — 42 B fixed overhead, no key or nonce on the wire. Bottom: multi-recipient rekey flood — the wrapped-key header of the preprint, confined to the rare control message; the body carries the coordinator-signed state $(e{+}1 \,\|\, gk_{e+1} \,\|\, R_{e+1} \,\|\, \sigma)$ under the one-time key k_{m}.

Algorithm 2 Receive data packet

Require: packet $P = H \,\|\, c$, candidate keys $\mathcal{K}$
1: parse H; **if** $\neg$REPLAYOK(src, seq, ts) **return** drop
2: **for** $K_{\mathrm{base}} \in \mathcal{K}$ **do** ▷ current, then bootstrap
3: $K_{\mathrm{pkt}} \leftarrow \mathsf{HKDF}(K_{\mathrm{base}}; -, src \,\|\, seq \,\|\, ts)$
4: $m \leftarrow \mathsf{Dec}_{K_{\mathrm{pkt}}}(0^{96},\ c,\ \bar{H})$
5: **if** $m \neq \perp$ **then return** m
6: **end if**
7: **end for**
8: **return** drop ▷ non-member or tampered

6 Anonymous Control Plane

6.1 Two-part Hello

Hellos (every 5 s, 2-hop scope) are the only messages partly readable by non-members, because the relay backbone — a connected dominating set with multipoint relays in the OLSR tradition [6] — is computed from them. A Hello has a cleartext public part (emitter pseudonym, sequence and timestamp, neighbor pseudonyms, and the elected MPR block) and a private part encrypted under gk_e carrying the true BD addresses, the emitter's X25519 public key, its Ed25519 signing key (private-part v0x03), and its current epoch. The *entire public part is the AAD of the private part*: an outsider can neither read the private fields nor forge acceptable public claims — tampering with the neighbor list or the MPR block invalidates the private tag. The private part uses a fresh random nonce (no *seq* available).

6.2 Network-wide key announcement

Hellos travel at most two hops, so a member never directly hears a distant peer's public key — and a *silent* member (one that emits no data) is never learned at all. Because pairwise keys, subgroup multicast, and the rekey flood all require the recipient's public key, missing keys would silently exclude members from private messaging *and from revocation*. Two designs were considered: embedding the sender's public key (group-encrypted) in every data packet, or a dedicated recurrent beacon. The former was rejected — +32 B on every packet forever, and silent members still never learned.

`PKT_ANNOUNCE` is a network-flooded beacon carrying the binding $(bd \to pk_{bd},\ pk^{\sigma}_{bd},\ e)$, encrypted exactly like a group data packet (Eq. 5, zero nonce, $\bar{H}$ as AAD), relayed and deduplicated by the existing broadcast machinery, and subject to the standard replay filter. Emission: a startup burst (3×, 2 s apart), then every 30 s, and once immediately after each epoch adoption. The v0x02 plaintext is *self-signed*: a 64-byte Ed25519 signature under the announced pk^{σ}_{bd} covers all other fields — a proof of possession (PoP) binding the ECDH key, the signing key, the address, and the epoch into one attested statement. Reception first verifies the PoP (invalid signatures are rejected and logged), then applies the **binding-conflict policy**: a key learned from a *direct Hello* private part is authoritative and never overwritten; otherwise the *first* announced binding is kept; later conflicting bindings are ignored with a `[SEC]` log. The receiver also checks that the announced *bd* equals the authenticated header *src*. The PoP eliminates mix-and-match forgeries (announcing someone else's ECDH key under one's own signing key, or vice versa); the residual insider-spoofing surface under derived identities is discussed in §10.

6.3 Ed25519 control-plane attestations

Signing key pairs are derived from the identity seed (Eq. 3), so no additional secret is provisioned, and verification keys are learned alongside ECDH keys through Hello v0x03 and `ANNOUNCE`. Two statements are attested: the `ANNOUNCE` binding (PoP above) and the rekey body (§7.2), where the signature makes the epoch transition attributable to its emitter and lets receivers enforce that the emitter is the coordinator.

7 Group Life Cycle

7.1 Epoch state and coordinator

Group state is the triple $(e,\ gk_e,\ R_e)$: monotone epoch, group key, revoked public keys. It is persisted atomically

(∼100 B; temp file then rename; mode `0600`; magic and version checked on load): a rebooted node resumes at its epoch, and a node with lost state falls back to e=0 and is healed as an absentee (§7.3). Epoch-based group rekeying echoes the epoch notion of MLS [7], reduced to a single shared key with no tree — adequate at bubble scale. The *coordinator* is the member with the smallest pseudonym among live members; there is no static leader to lose, and the convention re-elects a successor with zero messages.

7.2 Revocation as one multi-recipient flood

To revoke M, the coordinator bumps the epoch, samples a fresh group key, adds pk_M to R, signs the new state, and floods a single `PKT_REKEY_FLOOD` whose header wraps a one-time key k_{m} once per non-revoked recipient (Fig. 2, bottom). With state $s = e{+}1 \,\|\, gk_{e+1} \,\|\, R_{e+1}$ and coordinator signature $\sigma = \mathsf{Sig}_{sk^{\sigma}_{\mathrm{coord}}}(s)$, the packet for recipient j is built as:

$$
\begin{aligned}
{K_{\mathrm{w}}}^{j} &= \mathsf{HKDF}\big({K_{\mathrm{p}}}^{j};\ s_{\mathrm{w}},\ src \,\|\, seq \,\|\, ts\big),\\
blob_j &= \mathsf{Enc}_{{K_{\mathrm{w}}}^{j}}\big(0^{96},\ k_{\mathrm{m}},\ aid_j\big), \qquad (7)\\
c &= \mathsf{Enc}_{k_{\mathrm{m}}}\big(0^{96},\ s \,\|\, \sigma,\ A\big),\\
A &= \bar{H} \,\|\, \mu_{\mathrm{RF}} \,\|\, \mathrm{SHA256}\big(aid_1 \,\|\, blob_1 \,\|\, \cdots\big). \qquad (8)
\end{aligned}
$$

Three properties follow. (i) The per-recipient AAD aid_j in Eq. (7) binds each blob to its slot: a case cannot be replayed into another recipient's position. (ii) The case digest inside the body AAD (Eq. 8) seals the *whole recipient set*: tampering with, deleting, or inserting any case makes the body fail for *every* recipient — the absence of the revoked member's case is thereby cryptographically committed. (iii) The mode marker μ_{RF} domain-separates this construction from the byte-identical subgroup format, so neither can be reinterpreted as the other.

Pre-bump wrap epoch. Wrap keys use the pairwise keys of the *current* epoch e, derived *before* the bump: the implementation captures e and derives all ${K_{\mathrm{p}}}^{j}$ prior to updating state, then clears the pairwise cache. A recipient unwraps with its own current-epoch pairwise key; one already ahead (via gossip) simply fails to unwrap and is served by the unicast path.

MTU-safe partition. When the cases exceed one frame, the recipient set is *partitioned* into independent, self-contained packets — each with its own *seq*, *ts*, k_{m}, and (identical, individually re-encrypted) signed body — never wire-fragmented; there is no reassembly. With case size 56 B and signed body length $|s \,\|\, \sigma| = 102 + 32|R|$, the per-packet capacity on a transport of MTU M is

$$
n_{\max} = \left\lfloor \frac{\mathrm{M} - \rho - 1 - (102 + 32|R|) - \tau}{56} \right\rfloor, \qquad (9)
$$

Algorithm 3 Coordinator: revoke member M

Require: target M; state $(e, {gk}_e, R_e)$
1: **assert** coordinator; ${pk}_M$ known; ${pk}_M \notin R_e$
2: $T \leftarrow$ members $\setminus \{M\}$, dropping any without a known key
3: **for** $j \in T$: derive ${K_{\mathrm{p}}}^{j}$ *at epoch* e ▷ pre-bump
4: $(e{+}1,\ gk_{e+1},\ R_{e+1}) \leftarrow (e{+}1,\ \mathrm{rand}_{32},\ R_e \cup \{{pk}_M\})$; persist
5: $\sigma \leftarrow \mathsf{Sig}_{sk^{\sigma}}\big(e{+}1 \,\|\, gk_{e+1} \,\|\, R_{e+1}\big)$
6: **for all** T_i in MTU-safe partition of T (Eq. 9) **do**
7: build `REKEY_FLOOD` over T_i (Eqs. 7–8); broadcast
8: **end for**
9: clear pairwise cache; emit `ANNOUNCE`

capped by the format's maximum case count. At n=32, $|R|$=1: on Bluetooth Classic (M=1021), $n_{\max} = 15$, so revocation takes $\lceil 31/15 \rceil = 3$ flood packets; on BLE L2CAP CoC (M=512), $n_{\max} = 5$ and $\lceil 31/5 \rceil = 7$ packets — in every case far below the $n{-}1$ routed unicasts of the baseline, and each flood packet is relayed once per backbone node. Algorithm 3 states the coordinator procedure.

Receiver-side attestation and coordinator check. Adoption is gated by two verifications performed on the decrypted body. First, the Ed25519 signature must verify under the *sender's* registered signing key ($\mathsf{Vf}_{pk^{\sigma}_{src}}(s, \sigma) = 1$); a sender with no known signing key, or an invalid signature, is rejected with a `[SEC]` log. Second, the sender's pseudonym must be the minimum among the receiver's known live members — the coordinator convention, now enforced at the *receiver*, not merely asserted at the sender. Together these make an epoch transition attributable and coordinator-bound: a non-coordinator member cannot push a rekey even with valid pairwise keys. The residual trust caveats (identity strategy, partial membership views) are stated in §10.

7.3 Adoption and gossip catch-up

A recipient locates its case by pseudonym, unwraps k_{m}, decrypts the body, and *adopts* iff the received epoch is strictly greater than its own — strict monotonicity rejects replays of stale floods. Absent its case, or on any $\perp$, the node stays silent: the normal outcome for the revoked member and for members carried in another partition.

The flood need not reach everyone. Each Hello private part carries the emitter's epoch; an up-to-date member that sees a *stale, non-revoked* epoch replies with the current state over the pre-existing *unicast* `REKEY` gossip channel (unchanged from the baseline). The rekey body carries the *complete* state (e', gk', R'), so one catch-up suffices regardless of how many changes were missed; convergence is one Hello cycle (≈5 s). A revoked member

is never caught up: its key is in R', so no member gossips to it. The same flood-plus-gossip machinery serves member *addition* and periodic *key rollover* — both are an epoch bump, differing only in whether R grows. For urgent traffic during the brief propagation window, the à-la-carte `SUBGROUP` multicast carries a message immediately to a named subset with the same wrapped-case format.

8 Security Analysis

8.1 Goals

G1 Group confidentiality: only current, non-revoked members recover group plaintext. **G2 Integrity:** any modification of payload or immutable header fields is detected. **G3 Replay resistance:** captured packets cannot be re-processed. **G4 Private confidentiality:** member-to-member traffic is readable only by the endpoints (under the identity assumption of §10). **G5 Revocation:** after revoking M, no key held by M decrypts subsequent traffic, globally and consistently. **G6 Control-plane authenticity:** key bindings and epoch transitions are attributable to their emitter and, for rekeys, coordinator-bound at the receiver.

8.2 Two public keys are not enough

G4 rests on the computational Diffie–Hellman assumption on Curve25519. A member C observing $A \to B$ traffic knows $pk_A = sk_A \cdot G$ and $pk_B = sk_B \cdot G$ and needs $sk_A sk_B \cdot G$. The only available group operation on points is addition, and $sk_A \cdot G + sk_B \cdot G = (sk_A + sk_B) \cdot G$, which is not the target; extracting a scalar from a point is the discrete-log problem. Hence one of the two private keys is required — unconditionally against outsiders, and against fellow members under the file-persisted identity strategy (§10).

8.3 Adversary coverage and validation

Table 3 maps adversary capabilities to defending mechanisms; Table 4 maps each claimed property to the tests that exercise it, including the adversarial cases.

8.4 On the fixed nonce

The zero nonce (§2.3) is safe precisely because Eq. (5) makes each key single-use, satisfying GCM's uniqueness requirement on the (key, nonce) pair [5]. The *ts* term additionally covers counter reuse across reboots, and the salt separation of Eqs. (5) and (6) ensures the same AES key is never used for two messages under the zero nonce.

9 Overhead

Table 5 compares on-wire cost with the preprint's multi-key scheme at $n = 32$.

10 Assumptions and Limitations

Stated explicitly; none regresses relative to the preprint, and two pre-existed there in worse form (without detection or repair).

1. **Identity-anchored attestations (primary).** Under the file-less strategy (Eq. 2), any holder of the group key can re-derive any member's identity seed — hence its ECDH key, its pairwise keys, *and its Ed25519 signing key* (Eq. 3). Against a motivated insider, G4 and the attestations of G6 then raise the bar (the attacker must target a specific member's seed) without being cryptographic guarantees. The file-persisted random seed (`keys.py`, mode `0600`) restores full intra-group G4 and makes signatures unforgeable by fellow members — a configuration change, not a code change — consistent with the preprint's short-mission, initially-trusted model.

2. **Convention-based coordinator legitimacy.** The receiver-side coordinator check compares the sender's pseudonym to the minimum over the receiver's *known* live members. A node with a partial membership view (partition, eclipse, early boot) could accept a rekey from a signer that is not the global minimum. Strict epoch monotonicity and the signed, attributable body bound the damage: any accepted transition is traceable to an identified member.

3. **Propagation window.** For the seconds between decision and convergence, a not-yet-switched member may emit under the old key, readable by M; `SUBGROUP` mitigates for urgent traffic. The preprint's local repudiation had a permanent, undetectable analogue.

4. **No intra-epoch forward secrecy.** Compromise of gk_e exposes epoch-e traffic; consistent with the preprint's "keys not renewed during the mission". Epoch rotation bounds exposure, and periodic rollover is the same mechanism at no extra cost.

5. **Not fully stateless.** Epoch state is persisted (∼100 B, atomic, `0600`); an amnesiac node is healed like an absentee.

6. **Rekey-flood metadata.** Case count and recipient pseudonyms are cleartext: an observer learns that a rekey occurred and estimates group cardinality; no padding or decoys are added. The prior $n-1$ unicasts leaked less.

7. **Pseudonym recomputation coupling.** Reaching a member known only via `ANNOUNCE` recomputes its

Adversary / capability	Attempt	Defending mechanism (goal)
Outsider, passive	Observe all frames	Payloads and Hello private parts under gk_e; only pseudonyms and CDS structure visible; BD addresses and public keys never in cleartext (G1).
Outsider, active	Inject or modify frames	Tag over c and $\bar{H} \Rightarrow \perp$; forged Hello rejected since the public part is the private part's AAD (G2).
Any party	Replay a captured frame	Dual window (W=256) + timestamp bound (Δ=120 s) + relay dedup; seq, ts also bind K_{pkt} (G3).
Fellow member	Read $A \rightarrow B$ private traffic	CDH: two public keys insufficient (§8.2); caveat under derived identities, §10 (G4).
Fellow member	Spoof an `ANNOUNCE` binding	Ed25519 PoP over the full binding + header-*src* check + Hello-first precedence + first-seen policy + `[SEC]` logging; residual under derived identities, §10 (G6).
Fellow member	Push a forged rekey / fork the group	Body signature verified under the sender's registered key + receiver-side min-pseudonym coordinator check + strict epoch monotonicity (G6, G5).
Compromised insider (detected) / stolen device	Keep using held keys	Epoch bump: gk_e and M's pairwise keys stop decrypting; $pk_M \in R$ blocks all future catch-up (G5).
External relay	Drop, delay, duplicate	Cannot read or modify; duplication deduplicated; residual is local denial, inherent to wireless.

Table 3: Adversary model and defensive coverage.

pseudonym from the bootstrap PSK (Eq. 1); sound while all nodes use that PSK as the pseudonym key. A per-epoch pseudonym would instead require carrying the pseudonym inside `ANNOUNCE`.

8. **Ordering nit.** The post-revocation `ANNOUNCE` is emitted under gk_{e+1} concurrently with the flood; laggards cannot read it until they adopt. Harmless (the beacon is periodic), noted for completeness.

9. **Collusion.** A member relaying the current key to M defeats any revocation; irreducible in every shared-key scheme, including the preprint's.

11 Conclusion

The Bubble-Blue security layer meets the operational requirements of the originating preprint at a fixed 42-byte per-packet cost by replacing key *distribution* with key *derivation* throughout, by confining the multi-recipient wrapped-key header to a rare, self-healing epoch flood, and by attesting control-plane statements with signatures derived from the existing identity seed. The construction is implemented and tested end to end on both transports, including the adversarial cases of Table 3 (Table 4). The identified next increments — defaulting to file-persisted identity seeds to make attestations insider-proof, padding or batching rekey metadata, and intra-epoch forward secrecy via ephemeral ECDH — strengthen assumptions without altering the wire formats specified here.

References

[1] N. Achir and P. Jacquet. *Bubble Blue: secure opportunistic Bluetooth mesh bubbles*. arXiv:2509.00967, 2025.

[2] H. Krawczyk and P. Eronen. *HMAC-based Extract-and-Expand Key Derivation Function (HKDF)*. RFC 5869, 2010.

[3] A. Langley, M. Hamburg, and S. Turner. *Elliptic Curves for Security*. RFC 7748, 2016.

[4] S. Josefsson and I. Liusvaara. *Edwards-Curve Digital Signature Algorithm (EdDSA)*. RFC 8032, 2017.

[5] M. Dworkin. *Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC*. NIST SP 800-38D, 2007.

[6] T. Clausen and P. Jacquet. *Optimized Link State Routing Protocol (OLSR)*. RFC 3626, 2003.

[7] R. Barnes et al. *The Messaging Layer Security (MLS) Protocol*. RFC 9420, 2023.

Property	Representative tests
G1 outsider excluded	`test_phase1a` (intruder, hardware); pairwise wrong-key $\Rightarrow \perp$
G2 tamper detected	header/ciphertext tamper tests (data, announce); case tamper breaks *all* decrypts
G3 replay rejected	wire replay (hardware); pairwise replay; announce replay
G4 pairwise secrecy	symmetric A/B derivation; third-key $\Rightarrow \perp$
G5 revocation	revoked absent from cases and cannot unwrap; 2-hop flood adoption; stale-epoch rejection; offline member healed by unicast gossip; announce-only member included
G6 attestations	invalid `ANNOUNCE` signature $\Rightarrow$ `rejected_sig`; rekey from non-coordinator or with bad signature rejected (`[SEC]`)
Partition	determinism; union = targets, no duplicates; per-packet cap from real constants on both MTUs

Table 4: Property-to-test map (unit and integration suites; hardware scripts noted).

	Preprint [1]	This work
Per-packet key header	$16n = 512\,\mathrm{B}$	$0\,\mathrm{B}$
Fixed pkt overhead	$\rho + 16n$	$\rho + \tau = 42\,\mathrm{B}$
Typical chat packet	$\sim$600 B	$\sim$88 B
Fits BLE MTU	no	yes (both)
Initial keying	$O(n^2)$ out of band	1 PSK + ECDH
Add a member	n keys out of band	1 flood
Revoke a member	local slot zeroing	3 (BT) / 7 (BLE)

Table 5: Overhead at $n = 32$. The wrapped-key header is confined to the rare rekey flood.